

Spominki

Amaru kupuje spominke v trgovini za turiste. Obstaja N **tipov** spominkov. V trgovini je na voljo neskončno mnogo spominkov vsakega tipa.

Vsak tip spominka ima fiksno ceno. Spominek tipa i ($0 \leq i < N$) stane $P[i]$ kovancev, kjer je $P[i]$ pozitivno celo število.

Amaru ve, da so tipi spominkov razvrščeni v padajočem vrstnem redu glede na ceno, in da so cene spominkov različne. Natančneje, $P[0] > P[1] > \dots > P[N-1] > 0$. Poleg tega je uspel izvedeti vrednost $P[0]$. Žal Amaru nima nobenih drugih informacij o cenah spominkov.

Za nakup spominkov bo Amaru opravil več transakcij s prodajalcem.

Vsaka transakcija je sestavljena iz naslednjih korakov:

1. Amaru prodajalcu izroči določeno (pozitivno) število kovancev.
2. Prodajalec odnese te kovance v zadnjo sobo in jih položi na kupček, kjer Amaru kovancev ne more videti.
3. Prodajalec pogleda vsak tip spominka $0, 1, \dots, N-1$, eno za drugo, in sicer v tem vrstnem redu. Vsak tip pogleda **le enkrat** na transakcijo.
 - Ko pogleda spominek tipa i , in če je trenutno število kovancev v kupčku večje ali enako kot $P[i]$, potem
 - prodajalec vzame $P[i]$ kovancev iz kupčka in
 - na mizo položi spominek tipa i .
4. Prodajalec vrne Amaruju vse kovance, ki ostanejo v kupčku ter vse spominke, ki so na mizi.

Upoštevajte, da se pred začetkom transakcije na mizi ni nobenih kovancev ali spominkov.

Vaša naloga je, da Amaruju naročite, kako naj opravi toliko transakcij, da:

- v vsaki transakciji kupi **vsaj en** spominek, in
- do konca kupi **točno** i spominkov tipa i , za vsak i , kjer $0 \leq i < N$. Upoštevajte, da to pomeni, da Amaru ne sme kupiti nobenega spominka tipa 0.

Amaru ni treba zmanjšati števila transakcij na minimum in hkrati ima neomejeno zalogo kovancev.

Podrobnosti implementacije

Implementirati morate naslednjo proceduro.

```
void buy_souvenirs(int N, long long P0)
```

- N : število tipov spominkov.
- $P0$: vrednost $P[0]$.
- Ta funkcija se pokliče enkrat za vsak testni primer.

Zgornja funkcija lahko kliče naslednjo funkcijo, s katero Amaru opravi transakcijo:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : število kovancev, ki jih Amaru izroči prodajalcu.
- Funkcija vrne par. Prvi element para je polje L , ki vsebuje tipe kupljenih spominkov (v naraščajočem vrstnem redu). Drugi element je celo število R , ki pove, koliko kovancev je prodajalec vrnil Amaruju po zaključku transakcije.
- Zahtevano je, da $P[0] > M \geq P[N - 1]$. Pogoji $P[0] > M$ zagotavlja, da Amaru ne kupi nobenega spominka tipa 0, in $M \geq P[N - 1]$ zagotavlja, da Amaru kupi vsaj en spominek. Če ti pogoji niso izpolnjeni, bo vaša rešitev prejela sodbo `Output isn't correct: Invalid argument`. Upoštevajte, da v nasprotju s $P[0]$, vrednost $P[N - 1]$ ni podana v vhodnih podatkih.
- Funkcijo lahko za vsak testni primer pokličemte največ 5000-krat.

Vedenje ocenjevalnika **ni prilagodljivo**. To pomeni, da je zaporedje cen P določeno, preden se pokliče `buy_souvenirs`.

Omejitve

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ za vsak i velja $0 \leq i < N$.
- $P[i] > P[i + 1]$ za vsak i velja $0 \leq i < N - 1$.

Podnaloge

Podnaloga	Točke	Dodatne omejitve
1	4	$N = 2$
2	3	$P[i] = N - i$ za vsak i , kjer $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ za vsak i , kjer $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ za vsak i , kjer $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ za vsak i , kjer $0 \leq i < N - 1$.
6	33	Brez dodatnih omejitev.

Primer

Razmislite o naslednjem klicu.

```
buy_souvenirs(3, 4)
```

Obstaja $N = 3$ tipov spominkov in $P[0] = 4$. Opazimo, da obstajajo le tri možna zaporedja cen P : $[4, 3, 2]$, $[4, 3, 1]$, in $[4, 2, 1]$.

Predpostavimo, da `buy_souvenirs` pokliče `transaction(2)`. Recimo, da klic vrne $([2], 1)$, kar pomeni, da je Amaru kupil en spominek tipa 2 in mu je prodajalec vrnil 1 kovanec. Opazimo, da nam to omogoča sklepati, da je $P = [4, 3, 1]$, saj:

- Za $P = [4, 3, 2]$ bi `transaction(2)` vrnil $([2], 0)$.
- Za $P = [4, 2, 1]$ bi `transaction(2)` vrnil $([1], 0)$.

Nato `buy_souvenirs` lahko pokliče `transaction(3)`, ki vrne $([1], 0)$, kar pomeni, da je Amaru kupil en spominek tipa 1, in mu je prodajalec vrnil 0 kovancev. Do zdaj je v skupnem seštevku kupil en spominek tipa 1 in en spominek tipa 2.

Na koncu `buy_souvenirs` lahko ponovno pokliče `transaction(1)`, ki vrne $([2], 0)$, kar pomeni, da je Amaru kupil en spominek tipa 2. Upoštevajte, da bi tukaj lahko uporabili tudi `transaction(2)`. Na tej točki v skupnem seštevku ima Amaru en spominek tipa 1 in dva spominka tipa 2, kot je zahtevano.

Vzorčni ocenjevalnik

Oblika vhoda:

```
N
P[0] P[1] ... P[N-1]
```

Oblika izhoda:

```
Q[0] Q[1] ... Q[N-1]
```

Tukaj $Q[i]$ predstavlja končno število kupljenih spominkov tipa i , za vsak i , kjer $0 \leq i < N$.